"Throws: Nothing" should be `noexcept`

Document #:	P1656R2
Date:	2020-02-11
Project:	Programming Language C++
Audience:	Library Evolution Working Group

Reply-to:	Agustín Bergé <agustinberge@gmail.com>

> “Has the C++ community considered adding `noexcept` specifications to standard-library functions that (in practice) never throw?”

0. History

Changes from P1656R1:

— Change proposed policy wording as directed by LEWG.

Changes from P1656R0:

— Adjust contract discussion for their removal from C++20.

— Correct description of `nothrow` attribute.

— Add alternative proposed policies.

1. Introduction

A function that "Throws: Nothing" is a non-throwing function, and as such it ought to have a non-throwing exception specification: `noexcept`. The reason some functions continue to be specified as "Throws: Nothing" in a `noexcept` world is *contracts*; functions with preconditions are only defined for the subset of its potential inputs that satisfy them. These functions are said to have a *narrow contract*, while functions defined for all its potential inputs —that is, with no preconditions— are said to have a *wide contract*:

— A *wide contract* for a function or operation does not specify any undefined behavior. Such a contract has no preconditions: A function with a wide contract places no additional runtime constraints on its arguments, on any object state, nor on any external global state. Examples of functions having wide contracts would be `vector<T>::begin()` and `vector<T>::at(size_type)`.

— A *narrow contract* is a contract which is not wide. Narrow contracts for functions or operations result in undefined behavior when called in a manner

> that violates the documented contract. Such a contract specifies at least one precondition involving its arguments, object state, or some external global state, such as the initialization of a static object. Examples of functions having narrow contracts would be `vector<T>::front()` and `vector<T>::operator[](size_type)`.

The `noexcept` policy followed by the standard library —first adopted with [N3279] and later revised by [P0884]— specify that a function that the LWG agree cannot throw should be marked as unconditionally `noexcept` if they have a *wide contract*. No such provision is specified for functions with a *narrow contract*, which are specified as "Throws: Nothing" instead. Making a function `noexcept` in effect partially defines the undefined nature of a narrow contract, making it impossible for an implementation to exploit it by throwing an exception.

It is a common misconception that the policy supports standard library implementations that wish to diagnose contract violations by throwing exceptions. No known implementation actually wants this. The policy supports one implementation's method of testing its contract checks: installing a throwing violation handler and expecting an exception when deliberately violating function preconditions. This is a form of negative unit testing commonly known as death testing, except that it is implemented on top of the stack unwinding mechanism required for exception support, rather than the usual process termination mode whose cost has proven to be inviable to large companies like Bloomberg.

Contracts and `noexcept`

Contracts as a language feature introduce build levels, continuation modes, contract violation handlers, and even the possibility to throw to signal failure! It is almost as if they were designed to support this particular testing scenario, except it runs into the same complication it did back in C++11: `noexcept`.

> [N4820] **[dcl.attr.contract.check]/6** If a violation handler exits by throwing an exception and a contract is violated on a call to a function with a non-throwing exception specification, then the behavior is as if the exception escaped the function body. [*Note:* The function `std::terminate` is invoked. -*end note*] […]

The missing piece would be a dedicated *test* build level, in which an exception thrown from a precondition's violation handler (but no other) would be allowed to propagate into the immediate caller, performing regular stack unwinding in the process.

This was originally proposed in [N3248], along with the recommendation to specify functions with a *narrow contract* as `noexcept` as well, but it was ultimately rejected by EWG.

Contracts do bring a change in perspective, nevertheless; the decision of exploiting the undefined nature of a narrow contract by throwing an exception shifts, from a library implementor to a library user, from a library's test suite to any context in which it is used.

2. Motivation

`noexcept` is a contract, it is a guarantee for the caller that no exception will escape the function. It is one of the first contracts to be expressed in the core language, later further promoted to take part of the type system.

Users expect `noexcept` contracts from the standard library, and are surprised when they find the committee is not yet ready for that level of commitment. They may suspect at first there could be some exception they have not anticipated — dynamic memory allocation is often a suspect—, or that it is simply an oversight on the part of the committee. Their suggestions to mark these "Throws: Nothing" functions as `noexcept` often turns into a stronger requirement once they learn about our policies and rationale.

A conservative approach was appropriate at the 11th hour of C++0x's standardization cycle, as there was little experience with the feature. By now implementations have matured, and users and implementors have gained experience with the feature.

We should revisit our `noexcept` design policies, and adjust them as appropriate. We should make an effort to educate users if we expect them to follow our practices, or at the very least so that our practices are not misrepresented as the desire to diagnose undefined behavior via exceptions.

What follows is information gathered on aspects that are invoked as reasons for adding or removing `noexcept`.

Diagnosing Undefined Behavior

Putting an exception specification on a narrow contract arbitrarily limits what an implementor can do in order to respond to that violation. It partially defines what would otherwise be truly undefined behavior, as it makes throwing an exception practically impossible.

A disadvantage of using exceptions for diagnostic purposes is that any evidence of the failed precondition may be destroyed as stack unwinding takes place, making the violation harder to diagnose. Furthermore, the exception might be swallowed by some user `catch` clause, or lead to termination on its way up the stack when combined with the ever more prevalent use of `noexcept` in user and library code. For these diagnosing exceptions to be useful, they need to be carefully integrated into the codebase.

Much of the rejection towards the `noexcept` design policies —from regular users as well as more than a few committee members —, originates from the incorrect belief that this is the scenario the policies are meant to support.

Control Flow

Exceptions may sometimes be used as a control flow mechanism, causing a jump to a remote `catch` clause. It can be seen as the C++ counterpart of C's `setjmp`/`longjmp`, as it performs stack unwinding, calling destructors on its way up the stack.

Examples of this use case are `boost::thread_interrupted` and pthread's cancellation requests, as well as Bloomberg's specific assertion test driver, for which the policy provides support. Contract violation assertions are tested by installing a throwing assertion handler, and expecting exceptions when deliberately violating the contract of the function under test.

Other standard library implementors test their assertions via *death tests*, named like that because the contract violation assertions cause the process to die. (See https://github.com/google/googletest/blob/master/googletest/docs/advanced.md#death-tests)

libstdc++ tests assertions by expecting the test to fail at runtime by exiting with a non-zero status. It had been suggested that exceptions should be used in debug mode, so that test could simply check for the corresponding exception rather than termination. Such suggestion was rejected as it would have been a major inconvenience to its users. (See https://gcc.gnu.org/bugzilla/show_bug.cgi?id=23888)

libc++ tests assertions by `fork`ing the test process, and executes the undefined code in the fork while the parent waits for it to terminate with an expected result. It had experimented with a throwing debug mode for ease of testing, but that approach proved not to work, and has been removed since. Contract violations under `noexcept` functions led to the introduction of a `_NOEXCEPT_DEBUG` macro, removing `noexcept` in debug mode, which was found to be viral yet insufficient. Furthermore, the change in observable behavior for the users was undesirable. (See https://reviews.llvm.org/D59166)

Codegen Bloat

For the purposes of codegen bloat, the effect of `noexcept` is twofold. On one hand, it obviates the need for stack unwinding, helping to avoid some of the cost that exceptions cause when they are not being used. The reduction in potential bloat, for both caller and callee alike, is an attractive property for those who are not using `-fno-exceptions`. On the other hand, enforcement of `noexcept` can require transforming any escaping exception into a call to `std::terminate`.

> **[except.spec]/5** Whenever an exception is thrown and the search for a handler encounters the outermost block of a function with a non-throwing exception specification, the function `std::terminate` is called. […]

For the purposes of codegen, let an expression be said to be *non-throwing* when no exception will be emitted from it. This includes all expressions for which the `noexcept` operator yields `true`, but also any other expression for which the implementation can determine that no exception will escape. The extent to which an expression is considered to be non-throwing depends on the implementation and the level of optimization being used, and it is a quality of implementation issue. Approx:

— A call to a "Throws: Nothing" function that is not marked `noexcept` yet whose definition is visible to the implementation, is usually determined to be non-throwing if its body does not contain any potentially-throwing expressions.

— A call to an *opaque* "Throws: Nothing" function —one whose definition is NOT visible to the implementation— that is not marked `noexcept`, such as a call to a function defined in a separate TU, or a call to a `virtual` function whose dynamic type gets resolved at runtime, is not usually determined to be non-throwing.

Additionally, a function may be annotated with an implementation specific attribute as a guarantee that no exceptions escape from it, so that the implementation will assume all calls to such a function to be non-throwing. The GNU form of this attribute is `__attribute__((nothrow))`, or `[[gnu::nothrow]]` in C++ attribute form, and it is equivalent to a `noexcept` specification. The MSVC form is `__declspec(nothrow)` and, unlike a `noexcept` specification, this attribute does not enforce termination.

Furthermore, MSVC supports different exception handling modes, and in `/EHsc` mode it assumes that functions declared as `extern "C"` are non-throwing. Despite being non-conforming, the most commonly used exception handling mode is `/EHsc`.

For exposition, consider the following snippet representing the interaction between user code and a "Throws: Nothing" library function which the implementation cannot determine to be non-throwing:

```
//! system
extern "C" void sys(void*); // (0)

//! library
template <typename T>
void lib(T* ptr) { // (1)
  assert(ptr != nullptr); // Throws: Nothing
  return sys(static_cast<void*>(ptr));
}
```

```
//! user
struct Dtor { ~Dtor(); };

static void user() { // (2)
  Dtor d;
  /* ... */
  lib(&d);
}

//# external linkage
void f() { user(); } // (3)
```

The cost of exception handling is reflected in the need for stack unwinding codegen for (2, 3) in the case (1) would throw, which is known not to do from the specification, but is not reflected in the implementation.

Annotating any of (0) or (1) with a nothrow attribute causes identical codegen to that of building with exceptions turned off. Wrapping (1) in a user defined forwarding function that is annotated with a nothrow attribute has the same effect. For MSVC, specifying the `/EHsc` exception handling mode —rather than the conforming `/EHs` one— causes the same results.

Marking (1) with a `noexcept` specification causes stack unwinding codegen to disappear, but introduces codegen for `std::terminate` enforcement. For GCC, this is directly encoded in the exception handling table as a terminate personality. For Clang and MSVC, this results in an implicit `catch (...)` handler being injected. The existence of a try-region appears to prevent MSVC from inlining said function. Further marking (2) with a `noexcept` specification has no effect in this scenario.

For the user concerned about potential bloat from exception handling in contexts that make no use of exceptions, a good approach seems to be to mark user defined functions with a `noexcept` specification, and to replace all calls to "Throws: Nothing" library functions with wrappers annotated with the implementation specific nothrow attribute.

`noexcept`-correctness

A correctness school of thought focuses on the `noexcept` operator giving an accurate answer, rather than seeing the `noexcept` specifier as a contract. Under this model, whether to mark a function depends on whether the implementation does actually throw; therefore, "Throws: Nothing" functions ought to be marked `noexcept`.

Under this model, the standard specification would mark as `noexcept` every function for which the answer is independent of the implementation, and thus portable. This includes not only functions already specified as "Throws: Nothing", as well as those that are implicitly "Throws: Nothing" by the equivalent-to method

of description, but also those with no explicit nor implicit "Throws" clause — e.g. `std::basic_string_view` constructors—.

Functions for which a corresponding `std::is_nothrow_*` type trait exists deserve a special mention, as the adoption of such traits would suggest that an accurate answer is expected to be significant.

3. Existing practice

Implementations are allowed to mark non-throwing functions as `noexcept`:

> **[res.on.exception.handling]/5** An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing exception specification.

Furthermore, contractual violations result in undefined behavior, giving implementors control of how to respond to those violations:

> **[res.on.required]/1** Violation of any preconditions specified in a function's *Requires*: element results in undefined behavior unless the function's Throws: element specifies throwing an exception when the precondition is violated.
>
> **[res.on.required]/2** Violation of any preconditions specified in a function's *Expects*: element results in undefined behavior.

Implementations mark many "Throws: Nothing" functions as `noexcept`, as well as others that simply don't throw on their particular implementation, but not all. There does not seem to be any discernible pattern, implementors seem to be deciding on a case-by-case basis where strengthening is reasonable, with no stable criteria.

The debug modes for *libstdc++* and *libc++* do not throw. Debug mode for *MSVC* does not throw as a means to diagnose violations either, but it does perform allocations which can potentially throw; no special provision is done for allocations from within `noexcept` functions, including strengthened ones, leading to termination.

4. Conclusions

The `noexcept` specifier allows us to express a *contract* explicitly in the specification, in the actual interface code, even in the type system. Implementations are allowed to strengthen this contract, and do so.

The use of `noexcept` in an implementation allows to reclaim some of the *non-zero cost* that exceptions introduce, although it is insufficient as it still leaves room for use of a nothrow attribute.

We should not compromise the *design* of interfaces used by millions of users for one implementation's preferred method of negative unit testing.

5. Proposed Policy

> **Adopt Proposed Policy: Minimal**
> SF F N A SA
> 6 14 0 4 1
>
> **Adopt Proposed Policy: Untangled from Contracts**
> SF F N A SA
> 9 9 0 5 2
>
> **Adopt Proposed Policy: Untangled from Contracts, keeping C library bit**
> SF F N A SA
> 8 10 2 5 0
>
> **Adopting the last.**

a. No library destructor should throw. They shall use the implicitly supplied (non-throwing) exception specification.

b. Each library function ~~having a *wide* contract (i.e., does not specify undefined behavior due to a precondition)~~ that LEWG and LWG agree cannot throw, should be marked as unconditionally `noexcept`.

c. If a library swap function, move-constructor, or move-assignment operator is conditionally-wide (i.e. can be proven to not throw by applying the `noexcept` operator) then it should be marked as conditionally `noexcept`.

d. If a library type has wrapping semantics to transparently provide the same behavior as the underlying type, then default constructor, copy constructor, and copy-assignment operator should be marked as conditionally `noexcept` the underlying exception specification still holds.

e. No other function should use a conditional `noexcept` specification.

f. Library functions designed for compatibility with "C" code (such as the atomics facility), may be marked as unconditionally `noexcept`.

6. Acknowledgements

Alisdair Meredith and John Lakos for carefuly explaining the contents of [N3248] and [N3279] as well as the history behind them in detail.

Ben Craig for patiently assisting with the codegen bloat analysis.

Barry Revzin, Billy O'Neal, Chris Kennelly, Eric Fiselier, Herb Sutter, Howard Hinnant, JeanHeyd Meneide, Jonathan Wakely, Marshall Clow, Michael Park, Peter Dimov, Tim Song, Titus Winter, for discussing the subject with me.

Everyone who answered my inquires on their use of `noexcept` and their view of the standard library design policies.

References

[N3248] Alisdair Meredith and John Lakos. 2011. `noexcept` Prevents Library Validation. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3248.pdf

[N3279] Alisdair Meredith and John Lakos. 2011. Conservative use of `noexcept` in the Library. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3279.pdf

[N4820] Richard Smith. 2019. Working Draft, Standard for Programming Language C++. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf

[P0884] Nicolai Josuttis. 2018. Extending the `noexcept` Policy. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0884r0.pdf